Retries are a helpful pattern whenever the service downstream is momentarily unavailable or slow due to overloading, but it's likely to heal soon. In this case, you should limit the number of retries and use exponential backoff to prevent adding extra load on an already overloaded service. On the other hand, you shouldn't retry the request if the service fails with a recurrent error, such as if it's entirely down or returns an acceptable error like 404. The following section will show you how to define fallbacks when specific errors happen.

8.4.3 Fallbacks and error handling

A system is resilient if it keeps providing its services in the face of faults without the user noticing. Sometimes that's not possible, so the least you can do is ensure a graceful degradation of the service level. Specifying a fallback behavior can help you limit the fault to a small area while preventing the rest of the system from misbehaving or entering a faulty state.

In the previous discussion of timeouts, you already provided a fallback behavior if no response is received within the time limit. You'll want to include fallbacks in your general strategy to make your system resilient, and not just in a specific case like timeouts. A fallback function can be triggered when some errors or exceptions occur, but they're not all the same.

Some errors are acceptable and semantically meaningful in the context of your business logic. When Order Service calls Catalog Service to fetch information about a specific book, a 404 response might be returned. That's an acceptable response that should be addressed to inform the user that the order cannot be submitted because the book is not available in the catalog.

The retry strategy you defined in the previous section is not limited: it will retry the request as long as an error response is received, including